

Keecas Quarto Example

Table of contents

How to use keecas in a Quarto document	1
Basic Symbolic Math with Units	2
Pipe Commands Demonstration	2
Different LaTeX Environments	2
Cases Environment	2
Equation Environment with Labels	2
Verification Function	3
Subs get ordered	4
Check Function Templates	4
Custom Templates	5
Summary	5
Template Configuration	6
Test using a QuartoMarkdownBase64	6

How to use keecas in a Quarto document

This example demonstrates all the key features of **keecas** for symbolic and units-aware calculations in a Quarto document.

Note

If the rendering engine is **KaTeX** (i.e. Jupyter notebook), the label command `\label{}` will result in a **ParseError**. The latex code will work when rendering by **quarto render**, but since you may want to see the result before rendering the document, an option to disable the `\label{}` command is provided.

When rendering with **quarto** you can have two config:

- if `qmd` files, then everything is fine

- if ipynb files, then pass the `--execute` flag to `quarto render` to rerender the notebook

Basic Symbolic Math with Units

Define symbols and calculate basic engineering quantities:

$$F = 10 \text{ kN} \quad \text{applied force} \quad (1)$$

$$A = 50 \text{ cm}^2 \quad \text{cross-sectional area} \quad (2)$$

$$\sigma = \frac{F}{A} = 2.00 \text{ MPa} \quad \text{normal stress} \quad (3)$$

Pipe Commands Demonstration

Using pipe operators for functional composition:

$$x = 3 \text{ m} \quad (4)$$

$$y = 4 \text{ m} \quad (5)$$

$$d = x^2 + y^2 = 25.0 \text{ m}^2 \quad (6)$$

Different LaTeX Environments

Cases Environment

Equation Environment with Labels

Calculate beam deflection with cross-references:

$$\delta = \frac{5 q L^4}{384 E I} = 15.95 \text{ mm} \quad (7)$$

i Note

labels emitted by `show_eqn` are latex labels, therefore they need to be referenced with `\eqref{}` or `\ref{}` latex commands.

`?@eq-QUARTO_EXAMPLE-delta` will not work

The deflection calculated in ?? shows acceptable values for serviceability.

Verification Function

Using the `check` function for design checks:

$$\sigma_{Sd} = 20 \text{ MPa} \quad (8)$$

$$\tau_{Sd} = 15 \text{ MPa} \quad (9)$$

$$\sigma_{Rd} = 250 \text{ MPa} \quad (10)$$

$$\tau_{Rd} = 75 \text{ MPa} \quad (11)$$

$$\frac{\sigma_{Sd}}{\sigma_{Rd}} = 0.080 \quad [\leq 1.0 \text{ VERIFIED}] \quad (12)$$

$$\frac{\tau_{Sd}}{\tau_{Rd}} = 0.200 \quad [\leq 1.0 \text{ VERIFIED}] \quad (13)$$

The type of comparison in the `check` function can be specified, as well the value to be specified against.

$$a = 3 \quad [> 1 \text{ NOT VERIFIED}] \quad (14)$$

$$b = 4 \quad [> 2 \text{ VERIFIED}] \quad (15)$$

$$c = 5 \quad [\geq 3 \text{ NOT VERIFIED}] \quad (16)$$

$$d = 6 \quad [\geq 4 \text{ VERIFIED}] \quad (17)$$

$$f = 7 \quad [\neq 5 \text{ NOT VERIFIED}] \quad (18)$$

Subs get ordered

Using `pc.subs` will automatically order the substitutions pairs topologically, so you don't have to order them yourself (default `sympy` behavior). For example let's define some mappings in any order:

$$\left\{ \delta = \frac{5 q L^4}{384 E I} \right. \quad (19)$$

If we use the default behavior of `sympy`'s `subs` method (unsorted substitutions) we get:

$$\left\{ \begin{array}{l} x = (1 + a)^2 \\ y = 4 \\ a = 3 \end{array} \right. \quad \text{partially evaluated} \quad (20)$$

While the default behavior of `pc.subs` is to sort the substitutions:

$$\left\{ \begin{array}{l} x = 16 \\ y = 4 \\ a = 3 \end{array} \right. \quad \text{fully evaluated} \quad (21)$$

i Note

`pc.subs` also take care of sympyfing the object before subsituting. In the above example `a` is mapped to `3`, an `int`, therefore during dict comprehension the `int` is piped to `pc.subs`. If you would have used the `sympy` `subs` method directly, you would have get an error because the type `int` does not have `subs` method (the corrected dict comprehension you should look like `S(rhs).subs(...)`).

Check Function Templates

The `check` function supports customizable templates for different visual styles:

$$\text{Default : } [\leq 1.0 \text{ } \mathbf{VERIFIED}] \quad (22)$$

$$\text{Boxed : } \boxed{\leq 1.0 \checkmark \mathbf{VERIFIED}} \quad (23)$$

$$\text{Minimal : } \leq 1.0 \checkmark \quad (24)$$

$$a = 3 \quad (25)$$

$$b = 4 \quad (26)$$

$$d = \frac{\sqrt{a^2 + b^2}}{c} = 0.417 \quad (27)$$

$$c = a \, b = 12.000 \quad (28)$$

Custom Templates

You can also use completely custom templates:

$$\text{Pass : } \leq 1.0 \mathbf{PASS} \quad (29)$$

$$\text{Fail : } > 1.0 \mathbf{FAIL} \quad (30)$$

Summary

This example demonstrated:

- Basic symbolic math with units
- Pipe command usage (see Section)
- Different LaTeX environments (align, cases, equation)
- Label and cross-reference functionality
- Dataframe operations
- Verification functions
- Complex symbolic manipulations

All calculations in Section show that keecas provides a powerful interface for engineering calculations in Quarto documents.

Template Configuration

Templates can also be configured globally via TOML config files:

```
[check_templates.template_sets.minimal]
success = '${symbol}{rhs} \,\textcolor{green}{\checkmark}$'
failure = '${symbol}{rhs} \,\textcolor{red}{\times}$'
```

This allows you to set project-wide or user-wide styling for all check functions.

Test using a QuartoMarkdownBase64

```

\begin{align}
a &= 5\backslash\,\backslash\text{m} & \backslash\backslash[8pt] \\
b &= 12\backslash\,\backslash\text{m} & \backslash\quad\backslash\text{length b} & \backslash\backslash[8pt] \\
c &= a + b & \backslash\quad\backslash\text{total length c} \\
\end{align}

```

$${}^{\prime\prime}=\text{latex}$$

$$a = 5 \text{ m} \quad (31)$$

$$b = 12 \text{ m} \quad \text{length } b \quad (32)$$

$$c = a + b \quad \text{total length c} \quad (33)$$

“ “